

Variables Selection

Francisco J. Palmero Moya

August 24, 2023

Abstract

A learning algorithm must choose a meaningful subset of features to focus its attention on while ignoring the rest in the feature subset selection problem. It has become the focus of much research in areas of application for which datasets with tens or hundreds of thousands variables are available. In this document, we propose five different methods for variable selection applied to a synthetic dataset which was generated from a triple correlated XOR gate. The results aim to elucidate the most appropriate methods for the problem at hand.

1 The problem: synthetic dataset

The synthetic dataset has been generated from a triple correlated XOR gate. Let $X = \{X_i\}_{i=1}^5$ be five Boolean random variables, the target function $f(X) = Y$ is defined as

$$Y = X_1 \oplus X_2 \oplus X_3 \quad (1)$$

$$\equiv X_1 \oplus X_4 \quad (2)$$

where $\oplus$ denotes XOR and the instance space is such that $X_4 \equiv X_2 \oplus X_3$. The dataset D consists on 100 instances of these five random variables and the target, $\{Y\} \cup \{X_i\}_{i=1}^5$. The samples $x_i^{(j)}$ for each $i = 1, 2, 3, 5$ are randomly drawn from X_i according to a Binomial distribution such that¹

$$X_i \sim B(n = 1, p = 1/2), \quad \forall i = 1, \dots, 5 \quad (3)$$

A simple calculation leads to the conclusion that there are only 2^4 possible instances given the definition of X_4 as $X_2 \oplus X_3$, and they are equiprobable. The marginal distribution of Y yields $Y \sim B(n = 1, p = 1/2)$.

The variable X_1 is strongly relevant; variables X_2, X_3 , and X_4 are weakly relevant; and X_5 is irrelevant following the definitions given by Kohavi and John [5].

The XOR operator is symmetric and associative, resulting in a symmetry in our target under certain permutations. These symmetries are closely related to the fact that each variable has no prediction power when used individually, but can classify the classes when used with the other ones. In fact, the XOR problem is a special case of parity function learning² and is considered hard for many feature selection algorithms.

Thus, for example, any algorithm which does not consider functional dependencies between features fails on this task. Only context-dependent local feature selection methods (like Relief, or filter methods applied to vectors in a localized feature space region) seem to be able to deal with such cases. The present document shows five different algorithms for subset selection making use of the *FSinR* [2] and *caret* packages from R. Some clearly bad algorithm for this problem are chosen to show their limitations.

2 Feature selection algorithms

Feature ranking and feature selection algorithms may roughly be divided into three types: filters, wrappers and embedded [3]. Here we briefly introduce three filters, and one wrapper; along with a space dimensionality reduction technique as an alternative feature selection method.

¹The definition of X_4 as $X_2 \oplus X_3$ makes X_4 follow a Binomial distribution $B(n = 1, p = 1/2)$ as well.

²The parity function is a classic concept in the field of computational complexity and machine learning. It refers to a function that takes a sequence of binary inputs and outputs 1 if the number of 1s in the sequence is odd, and outputs 0 if the number of 1s is even. In other words, the parity function checks whether the input sequence has an odd or even number of 1s and returns a corresponding output.

2.1 The filter approaches

These filter methods are based on performance evaluation metric calculated directly from the data, without direct feedback from predictors that will finally be used on data with reduced number of features.

The feature filter is a function returning a relevance measure $J(S|D)$ that estimates, given the dataset D , how relevant a given feature subset S is for the target Y . Since the dataset and the target are usually fixed and only the subsets S vary, the relevance measure may be written as $J(S)$.

Relevance measures may be computed for individual features X_i , $i = 1, \dots, N$ providing indices that establish a ranking order $J(X_{i_1}) \leq J(X_{i_2}) \leq \dots \leq J(X_{i_N})$. For independent features this may be sufficient, but if features are correlated many of important features may be redundant. Ranking does not guarantee that the largest subset of important features will be found. In such cases, a subset selection would be needed.

In order to make a search for good variable subsets, an evaluation mechanism for an individual variable subset needs to be defined first making search methods independent of the evaluation of feature subsets. Once an evaluation method such as relevance measures for the variable subsets has been defined, one can start the search for the best subset. A search strategy defines the order in which the variable subsets are evaluated.

The selected filter approaches are closely related to the problem at hand, and given the non-linearity and high correlation of the problem, most of them are based on subset selection and imply complex search algorithm in order to obtain good results. These filters are: a family of filters called Relief-based feature selection algorithms (RBAs) [4] based on correlation; the Information Gain Ratio (IGR) [9] based on information theory; and the Sufficient test from FOCUS [1] based on inconsistency. IGR and Sufficient test are both subset measures and, therefore, make use of search algorithm; but Relief is an individual measure.

Relief Relief operates by assigning relevance weights to features based on how well the value of a given feature helps to distinguish between instances that are near to each other. It attempts to find all relevant features. The original Relief samples n (user parameter) instances $x^{(i)}$ randomly from the training set and updates the relevance values based on the difference between the selected instance and the nearest instances of the same (near-hit $h^{(i)}$) and opposite (near-miss $m^{(i)}$) class³. Thus, the relevance update is

$$J_R(X_j) \leftarrow J_R(X_j) + \frac{1}{n} \left[\text{diff}(x_j^{(i)}, m_j^{(i)}) - \text{diff}(x_j^{(i)}, h_j^{(i)}) \right] \quad (4)$$

where diff stands for the Euclidean distance for continuous attributes; and either 1 (the values are different) or 0 (the values are equal) for discrete attributes. The range of values given the relevance measure $J_R(X)$ of relief is then $J(X_j) \in [-1, 1] \subset \mathbb{R}$ for each $j = 1, \dots, N$.

The original Relief algorithm has since inspired a family of RBAs [11], where some changes are applied to improve the scope or performance. Kononenko, Šimec, and Robnik-Šikonja [6] proposed ReliefF to increase the reliability of the probability approximation. ReliefF searches for k -nearest neighbors (KNN) hit/misses instead of only one near hit/miss and averages their contributions. Finding nearest neighbors assures that the feature weights are context-sensitive, but are still global indices. In addition, it also has some advantages dealing with incomplete data and multi-class problems (see footnote 3). Finally, Kononenko, Šimec, and Robnik-Šikonja [6] presented results where diff in (4) and distances in the KNN search are Manhattan-distance instead of Euclidean. Nevertheless, they claim that results are practically the same.

As a summary, the key idea behind Relief is to estimate feature relevance according to how well feature values distinguish target concept Y values among instances that are similar (near) to each other. This idea translates into a probability interpretation of the relevance measure as

$$J(X_j) \simeq \Pr(X_j \neq x_j^{(i)} \mid m_j^{(i)}) - \Pr(X_j \neq x_j^{(i)} \mid h_j^{(i)}) \quad (5)$$

As derived by Kononenko, Šimec, and Robnik-Šikonja [6] the “nearest instance” condition and the resulting fact that Relief weights are averaged over local estimates in smaller parts of the instance subspace (rather than globally over all instances) that enables Relief algorithms to take into account the context of other features and detect interactions.

³Original Relief assumes a binary class problem. Kira and Rendell [4] claim that Relief can be used in datasets with more than two classes by splitting the problem into a series of 2-class problems. Nevertheless, this solution seems unsatisfactory [6].

Information Gain Ratio Information theory is a field of study that deals with quantifying and analyzing the information content of data. It provides measures to capture the amount of uncertainty associated with different variables. In the context of feature selection, information theory can help us quantify how much information a particular variable contributes to predicting the target variable. The key concepts from information theory that are relevant here are entropy and mutual information:

Entropy measures the amount of uncertainty or randomness in a set of values. Its definition for a discrete random variable X is

$$H(X) = - \sum_{x \in X} \Pr(X = x) \log_2 \Pr(X = x) \quad (6)$$

where the base of the logarithm determines the information units (base 2 implies bits).

Mutual information quantifies the amount of information shared between two variables. It measures how much knowing the value of one variable reduces the uncertainty about the other variable.

$$MI(Y, X) = H(Y) + H(X) - H(Y, X) \quad (7)$$

where $H(Y, X)$ is the information contained in the joint distribution of X and Y

$$H(Y, X) = - \sum_{x \in X} \sum_{y \in Y} \Pr(Y = y, X = x) \log_2 \Pr(Y = y, X = x) \quad (8)$$

Mutual information is used to assess the relationship between two variables and is an important concept in feature selection. A feature is more important if the mutual information between the target and the feature distribution is larger.

Decision trees use a closely related quantity called information gain (IG). In the context of variable selection, this gain is simply

$$IG(Y, X) = H(Y) - H(Y|X) \quad (9)$$

between information contained in the target distribution $H(Y)$, and information after the distribution of variable X is taken into account, that is the conditional information $H(Y|X)$. A simple derivation shows that, in the context of feature selection⁴, information gain is nothing but the mutual information previously defined because

$$H(Y|X) = H(Y, X) - H(X) \quad (10)$$

Various modifications of the information gain have been considered in the literature on decision trees [10], particularly the information gain ratio (IGR) defined as the mutual information with a normalization factor

$$IGR(Y, X) = \frac{MI(Y, X)}{H(X)} \quad (11)$$

The normalization factor $H(X)$ is introduced to mitigate the bias induced by IG which tends to favor features with more values (attributes) since they inherently carry more information.

Note how information base relevance measures can be used for ranking individual variables or subsets, given their definitions. The definitions are expressed only in terms of discrete random variables because is our case, but they can be easily transformed into continuous variables. The main problem in the continuous case is how to empirically estimate the probability density functions, since they are unknown a priori. In the discrete case, the problem is easier because the probabilities are estimated from frequency counts.

Consistency The idea of inconsistency or conflict, a situation in which two or more vectors with the same subset of feature values are associated with different classes, leads to a search for subsets of features that are consistent [1]. The inconsistency count is equal to the number of samples with identical features, minus the number of such samples from the class to which the largest number of samples belong. Summing over all inconsistency counts and dividing by the number of samples the inconsistency rate for a given subset is obtained. This rate is an interesting measure of feature subset quality, for example it is monotonic, decreasing with the increasing feature subsets.

⁴ IG is primarily used in decision tree algorithms, specifically when deciding how to split the data at each node of the tree. These splits are usually not based in all attribute values and thus IG in general is different from MI .

Features may be ranked according to their inconsistency rates, but the main application of this index is in subset selection. The output of the sufficient test is either 1 if the subset is sufficient to construct a consistent hypothesis or 0 otherwise. The goal is then finding the smallest subset having 1 as output.

2.2 Space dimensionality reduction

If the dimensionality of the data is very high, some techniques might be used to project or embed the data into a lower dimensional space while retaining as much information as possible. One classical example is Principal Component Analysis (PCA). The coordinates of the data points in the lower dimension space might be used as features or simply as a means of data visualization.

The underlying principle of PCA revolves around finding the orthogonal axes (principal components) along which the data varies the most⁵. These axes are ranked based on the variance of the data projected onto them, with the first principal component capturing the maximum variance, the second capturing the second maximum, and so forth.

Mathematically, PCA involves computing the covariance matrix of the original data and then performing eigendecomposition on this matrix to obtain the eigenvectors and eigenvalues⁶. The eigenvectors, representing the directions of maximum variance, become the new coordinate axes in the lower-dimensional space, and the corresponding eigenvalues quantify the amount of variance retained along each axis. Picking the k eigenvectors having greater variance (eigenvalue) could be understood as a feature selection method.

PCA is inherently a linear technique, and it aims to capture linear correlations between variables. If the underlying relationships within the data are nonlinear, PCA might not accurately represent the data.

2.3 The wrapper approaches

Filters and wrappers differ mostly by the evaluation criterion. It is usually understood that filters use criteria not involving any learning algorithm, e.g., a relevance index based on correlation coefficients or test statistics, whereas wrappers use the performance of a learning algorithm trained using a given feature subset. A common choice for performing the evaluation in a wrapper method is cross-validation

Search strategies are the same as filters, however, if the evaluation scheme utilizes the particular predictor architecture for which the variables are being selected, then we have a wrapper approach.

The evaluation criteria for subsets selection chosen was the accuracy of a random forest model associated with a genetic algorithm as a search strategy.

Random Forest + Genetic Algorithm A random forest⁷ is an ensemble machine learning model that combines multiple decision trees to improve predictive accuracy and control overfitting. The final predictions are determined through a majority vote or averaging of the predictions from individual trees. On the other hand, a genetic algorithm (GA) is a heuristic search technique inspired by the process of natural selection. It aims to find solutions to complex problems by mimicking the principles of genetics and evolution.

The genetic algorithm uses a local encoding as a binary representation of the population. That is, any individual is a string of 1s and 0s where the i -th input is 1 if the variable X_i is selected and 0 otherwise. The fitness function is defined as the accuracy of the random forest that results when it has variables represented in the individual local encoding as model inputs. Some other parameters such as number of generations (iterations), crossover rate, mutation rate, etc. are user defined.

The model evaluation is carried out by a K -fold cross-validation. There are two kinds of performance: internal (during training) and external (during testing). After the model validation, the best parameters based on external validation are chosen to run a final search using the entire dataset. That means, the genetic algorithm runs k times before we get a final result concerning selected variables.

Kudo and Sklansky [8] explicitly recommend that GAs should be used for large-scale problems with more than fifty candidate variables. GA takes much more time than the others in smallscale and medium-scale problems. This is because GAs needed many iterations (generations) for their convergence. On the contrary, in large-scale problems, this impacts on execution time and makes GA more efficient [8, Section 4.4.3].

⁵PCA assumes that the variance of the data corresponds to its importance, which might not hold in all cases.

⁶The principal component transformations can then be associated with the singular value decomposition.

⁷Please note that random forest algorithm is used only as a learning algorithm that returns an accuracy for a given subset.

3 Rationale for Selected Feature Selection Approaches

In the previous section, we delved into the fundamentals of the XOR problem and explored three filters and one wrapper used for feature selection. Now, in this section, we delve deeper into the reasoning behind our selection of these specific approaches.

The idea behind the selected filters is to choose methods based on different kinds of measures for feature relevance. Then, we choose ReliefF whose measure is based on correlation, IGR whose measure is based on information and FOCUS whose measure is based on consistency.

The XOR problem has an interesting property: a variable irrelevant by itself can be relevant with others [3]. Therefore, individual measures of relevance, that do not take into account the context⁸, are useless in this problem. The Relief method is a classical example where it uses multivariate criteria to rank individual features, therefore according to their relevance in the context of others. Particularly, we use the ReliefF implementation where we have two user parameters: neighbors count and sample size. The main disadvantage of Relief is that it tries to find all relevant variables, therefore does not help with redundant features. As stated by Kohavi and John [5, p. 281], weakly relevant features will not be removed by Relief if they have high correlation with the target.

The Information Gain Ratio is introduced to precisely show that context-dependence. *IGR* is then computed as both subset and individual measure to illustrate it. The *IGR* method is parameter free, avoiding any bias that may be introduced by user parameters. The employed search algorithm is the well known hill-climbing⁹ where a local encoding is used. The local encoding allows us to check the *IGR* during the search for each evaluated subset.

Finally, the sufficient test is included to introduce an interesting topic such as sufficiency. Sometimes, we are only interest in finding a subset of features that is able to yield the same conclusions as the original set of variables. If the selected subset has lower dimensionality, the learning algorithm applied after feature selection could reach better efficiency. The low complexity of the problem with only five variables, allow us to run an exhaustive search over the entire dataset and ensure we have indeed the minimum sufficient subset. The search strategy employed is the Depth-First and Breadth First Search [7] that search in linear time.

The non-linearity of the problem make some approaches like PCA useless. The principal components from PCA are linear combinations of the original features, and we know that the data space cannot be expanded in terms of linear variables *per se*. Furthermore, it is important to note that PCA does not inherently function as a variable selection method, as the chosen features align with a novel variable space.

Finally, a high-complex search algorithm such as GA is introduced to show the broad range of possibilities available. The inherent representation of individuals as local encoding of subsets, and their fitness as the accuracy of the given learning algorithm (random forest) illustrate the process of wrappers for feature selection as a nice combination of evaluation methods associated with a search algorithm. Its use also highlights the notion of *using a sledgehammer to crack a nut*, where some simple solutions may be sufficient to solve the problem at lower cost.

4 Results

This section provides an overview of the obtained results, including the configurable parameter values utilized for each technique. It addresses the justification for using default values and summarizes outcomes when various parameter values were tested.

⁸Context dependence may include different relevance for different classes, and different relevance in different areas of the feature space

⁹The algorithm begins with an initial subset of features and iteratively explores neighboring subsets by adding or removing individual features. If a neighboring subset improves the relevance/performance, the algorithm moves to that subset and continues the process. This iterative procedure continues until no further improvements can be made in the evaluation metric.

4.1 Filters

4.1.1 Relief

ReliefF is the algorithm implemented in *FSinR* library from R. The execution with default parameters of neighbors count ($k = 5$) and sample size ($n = 10$), yields a relevance measure defined as (4),

X_1	X_2	X_3	X_4	X_5
0.96	0.02	0.02	0.04	-0.02

Table 1: ReliefF results for $k = 5$ and $n = 10$.

The results for different values of k and n yields similar results. The relevance of X_1 is always close to 1, X_2 and X_3 has similar relevance around 0.2, and X_4 is approximately 0.4. The variable X_5 has always relevance close to 0, sometimes taking negative values. If $k = 1$, as the original Relief, while keeping $n = 10$, then $J(X_i)$ is null for all variables except for $i = 1$ where $J(X_1) = 1$.

4.1.2 Information Gain Ratio

The *IGR* measure is implemented in *FSinR* library including the option of select the desired search strategy. The Table 2 shows the results of the application of the hill-climbing algorithm for *IGR* measure of relevance

Vector	Fitness	End?
00100	0.000246	no
00101	0.018164	no
00111	0.0239137	no
10111	0.2550662	no
10011	0.3355341	no
10010	0.5019174	no
10011	0.3355341	yes

Table 2: Information Gain Ratio results.

The *IGR* relevance measure is parameter free, meaning that multiple measures of the same subsets would have the same values. The high-climbing algorithm for its part has some parameters such as: initial vector with the initial subset of features, number of neighbors to evaluate at each iteration, and number of repetitions of the algorithm. Since we do not provide any initial vector its starts at random. Additionally, the repetitions is 1 as default and the number of neighbors is the default option of all possibles neighbors.

We also run the algorithm for individual measures as subsets with only one variable

10000	01000	00100	00010	00001
0.0001687684	0.0001687684	0.004296881	0.0003389961	0.01212898

Table 3: Information Gain Ratio for individual variables.

4.1.3 FOCUS

The last algorithm is the FOCUS algorithm implemented in *FSinR* library as binary consistency. The search strategy is the Depth-First and Breadth First Search. Evaluation and search strategy are both parameter free.

The result is the subset of X_1 and X_4 , without any relevance measure as explained before. The hill-climbing algorithm is also compatible with the consistency evaluation, therefore we decided to run it as well and check the results. We saw that any subset containing X_1 and X_4 , or X_1, X_2 and X_3 is sufficient. Other subsets like X_1 and X_2 or X_1 and X_3 are not sufficient.

4.2 PCA

Table 4 shows the importance of each component after the PCA.

	Comp. 1	Comp. 2	Comp. 3	Comp. 4	Comp. 5
Standard deviation	0.5441079	0.5161424	0.4978374	0.4727866	0.4586658
Proportion of Variance	0.2379468	0.2141159	0.1991979	0.1796553	0.1690840
Cumulative Proportion	0.2379468	0.4520627	0.6512607	0.8309160	1.0000000

Table 4: Importance of components in PCA

On the other hand, Table 5 shows the linear transformation between coordinate systems

	Comp. 1	Comp. 2	Comp. 3	Comp. 4	Comp. 5
X_1	0.465	0.327	0.338	0.743	0.103
X_2	-0.317	-0.182	0.880	0	-0.294
X_3	0.103	0.810	0	-0.370	-0.442
X_3	-0.604	0	-0.311	0.551	-0.478
X_3	-0.555	0.444	0.123	0	0.692

Table 5: Loading in PCA

4.3 Wrapper

4.3.1 Random Forest + Genetic Algorithm

The feature selection using GA was done making use of the *caret* library from R. The model evaluation consist on a random forest accuracy and a 5-fold cross-validation. The parameters for the GA are setting as: maximum number of generation to 50, population size¹⁰ to 32, crossover-rate to 0.7, mutation rate to 0.1 and elitism¹¹ to only 1.

During resampling the top 5 selected variables (out of a possible 5) where

X_1	X_2	X_3	X_4	X_5
100 %	40 %	40 %	60 %	20 %

Table 6: Random Forest + GA. Validation

Finally, in the search using the entire training set 4 features where selected at iteration 1 including: X_1 , X_2 , X_3 , and X_4 . The model performance in the test set was: accuracy equals 1 and kappa equals 1.

5 Discussion

In this section, we discuss the results obtained. The discussion is again split into different subsection based on the approach.

5.1 Filters

5.1.1 ReliefF

The subset selection based on Table 1 works as follows: if we rank the relevance measures and directly select the k best values we would always take X_1 and X_4 for any $k \geq 2$. We can observe that X_2 and X_3 have both the same relevance but is close to zero. The variable X_5 is discarded because its negative value¹².

¹⁰Note that the population size is set to the maximum number of different solutions for a 5-digits binary string, but it does not imply that all possible solutions are evaluated at each iteration since there may be duplicates.

¹¹Elitism is the number of the best subsets to survive at each generation

¹²Same approach than Kohavi and John [5] for negative relevance values in ReliefF.

There are other selection criteria such as percentile that seems more adequate for the given distribution of relevance values. Although X_1 and X_4 have the two highest values, their relevance is significantly different. The measure range is between -1 and 1, therefore whereas X_1 is close to the maximum value, X_4 is closer to zero than one.

As a summary, different selection criteria end up with different subsets where all of them share X_1 for any non-empty subset and may or may not include X_2 , X_3 and X_4 .

5.1.2 Information Gain Ratio

The results for *IGR* show the highest relevance value when the local encoding is 10010, that is, for X_1 and X_4 as final subset. In this case, the difference between the highest value and the second one is bigger.

It is interesting to check in Table 3 how individual features have an *IGR* value close to zero for all the variables. Indeed, this shows how irrelevant individual variables becomes relevant when they are together.

If we assume that *IGR* truly represents the relevance of a subset of measures, we can argue¹³ that $S = \{X_1, X_4\}$ is the most relevant subset given our data. Nevertheless, it does not imply that any learning algorithm trained only on S will have the same performance as trained on X since relevance does not imply optimality [5, Example 3].

5.1.3 FOCUS

We run an exhaustive search over all the possible subsets and the result was: the smallest subset that pass the sufficiency test is $S = \{X_1, X_4\}$. Obviously, since S is sufficient, any variable added to S keep the subset sufficient. The interested point here is that $S = \{X_1, X_2, X_3\}$ also pass the test, although its size is bigger, that is why we finally select only X_1 and X_4 as subset.

5.2 PCA

The results from Table 4 show similar proportion of variance values¹⁴, i.e., similar importance for each new component. This implies that we cannot choose a lower dimensionality space without a serious loss of relevance/importance.

If we would like to select a lower dimensionality space, even with four components, the cumulative proportion is around 80 %. It means that a 20 % of variance¹⁵ is lost only for reducing by one the space dimensionality. Since the proportion of variance is similar, for each dimension we would like to reduce our space, the resulting space will represent 20 % less of variance.

Table 5 shows how to express the new variables as a linear combination of the old ones, and vice versa. It corresponds to a linear transformation between variables spaces. If we decide to select the components from PCA as the variables' representation, any given instance must be transformed according to these combinations.

5.3 Wrapper

5.3.1 Random Forest + Genetic Algorithm

The results obtained from RF + GA are more complex than the previous one because they involve validation and a more complex search. The 5-fold cross-validation is used for hyperparameter tuning, i.e., set parameters such as maximum number of generations for the GA. The caret library does not provide these hyperparameters, therefore we cannot discuss them.

Table 6 shows the results during hyperparameter tuning in terms of selected variables. The key point here is the fitness evaluation that refers to the random forest accuracy. It seems like the accuracy is always one (maximum value) after running the GA, therefore redundant variables cannot be directly removed.

¹³By abusing of our condition as generative model creators, we have checked the *IGR* value for $S = \{X_1, X_2, X_3\}$ being it lower. An exhaustive search would be needed to safely check it for all possible subsets, and although is possible in our simple problem, is not the case in most real situations.

¹⁴The proportion of variance here refers to the weights of each eigenvalue given the sum of them.

¹⁵Remember that PCA assumes that the variance of the data correspond to its importance.

Indeed, running the same algorithm¹⁶ in *FSinR*, where the results at each step are available, we can check how for 10 repetitions of the GA the best value (fittest) was always 1.0 and the mean fitness value was always around 0.8. We are facing a problem of overfitting, which seems reasonable given the simple dataset and the complex feature selection algorithm.

As a summary, we should select a subset containing the minimum number of variables which fitness value was one. Therefore, we maximize our measure of accuracy while minimize the size of the subset.

6 Conclusions

A study of different feature selection techniques have been described. The approach followed started in Section 1 by defining a generative model (the XOR problem) to obtain a synthetic dataset where we can apply some feature selection methods described in Section 2. As has been shown in Section 3, each method has its own limitations and scopes. The ReliefF filter seems to be the best individual measure fit for the XOR problem given its context dependence. Other filters such as *IGR* shows the key property of some variables being relevant together but not individually. The use of subset measures requires search strategies such as hill-climbing, used in this document. These search strategies may vary in complexity, for example, for the sufficiency test (FOCUS) the search was done in the entire variable space because of its simplicity in our case. In other cases where this exhaustive search is not possible, there exist other alternatives such as GA. A basic GA along with a random forest as evaluation is the selected wrapper for our study. Finally, the PCA has been studied, and it has clearly demonstrated its inefficacy in non-linear problems as XOR.

References

- [1] Hussein Almuallim and Thomas G. Dietterich. "Learning Boolean concepts in the presence of many irrelevant features". In: *Artificial Intelligence* 69.1 (1994), pp. 279–305. ISSN: 0004-3702. DOI: [https://doi.org/10.1016/0004-3702\(94\)90084-1](https://doi.org/10.1016/0004-3702(94)90084-1). URL: <https://www.sciencedirect.com/science/article/pii/0004370294900841>.
- [2] F. Aragón-Royón et al. *FSinR: an exhaustive package for feature selection*. 2020. arXiv: 2002.10330 [cs.LG].
- [3] Isabelle Guyon and Andre Elisseeff. "An introduction to variable and feature selection". In: *J. Mach. Learn. Res.* 3 (2003), pp. 1157–1182. ISSN: 1533-7928. URL: <http://portal.acm.org/citation.cfm?id=944919.944968>.
- [4] Kenji Kira and Larry A. Rendell. "A Practical Approach to Feature Selection". In: *Machine Learning Proceedings 1992*. Ed. by Derek Sleeman and Peter Edwards. San Francisco (CA): Morgan Kaufmann, 1992, pp. 249–256. ISBN: 978-1-55860-247-2. DOI: <https://doi.org/10.1016/B978-1-55860-247-2.50037-1>. URL: <https://www.sciencedirect.com/science/article/pii/B9781558602472500371>.
- [5] Ron Kohavi and George H. John. "Wrappers for feature subset selection". In: *Artificial Intelligence* 97.1 (1997). Relevance, pp. 273–324. ISSN: 0004-3702. DOI: [https://doi.org/10.1016/S0004-3702\(97\)00043-X](https://doi.org/10.1016/S0004-3702(97)00043-X). URL: <https://www.sciencedirect.com/science/article/pii/S000437029700043X>.
- [6] Igor Kononenko, Edvard Šimec, and Marko Robnik-Šikonja. "Overcoming the Myopia of Inductive Learning Algorithms with RELIEFF". In: *Applied Intelligence* 7.1 (Jan. 1997), pp. 39–55. ISSN: 0924-669X. DOI: 10.1023/A:1008280620621. URL: <https://doi.org/10.1023/A:1008280620621>.
- [7] Dexter C. Kozen. "Depth-First and Breadth-First Search". In: *The Design and Analysis of Algorithms*. New York, NY: Springer New York, 1992, pp. 19–24. ISBN: 978-1-4612-4400-4. DOI: 10.1007/978-1-4612-4400-4_4. URL: https://doi.org/10.1007/978-1-4612-4400-4_4.
- [8] Mineichi Kudo and Jack Sklansky. "Comparison of algorithms that select features for pattern classifiers". In: *Pattern Recognition* 33.1 (2000), pp. 25–41. ISSN: 0031-3203. DOI: [https://doi.org/10.1016/S0031-3203\(99\)00041-2](https://doi.org/10.1016/S0031-3203(99)00041-2). URL: <https://www.sciencedirect.com/science/article/pii/S0031320399000412>.

¹⁶The library *FSinR* use the models given by *caret* as wrappers evaluators.

- [9] J. R. Quinlan. "Induction of decision trees". In: *Machine Learning* 1.1 (1986), pp. 81–106. DOI: 10 . 1007/BF00116251. URL: <https://doi.org/10.1007/BF00116251>.
- [10] J. R. Quinlan. "Induction of decision trees". In: *Machine Learning* 1.1 (1986), pp. 81–106. DOI: 10 . 1007/BF00116251. URL: <https://doi.org/10.1007/BF00116251>.
- [11] Ryan J. Urbanowicz et al. "Relief-based feature selection: Introduction and review". In: *Journal of Biomedical Informatics* 85 (2018), pp. 189–203. ISSN: 1532-0464. DOI: <https://doi.org/10.1016/j.jbi.2018.07.014>. URL: <https://www.sciencedirect.com/science/article/pii/S1532046418301400>.